

SecureVault

Zero-knowledge encrypted file vault

Complete project documentation · MERN + Prisma

Jo Alex · October 2026

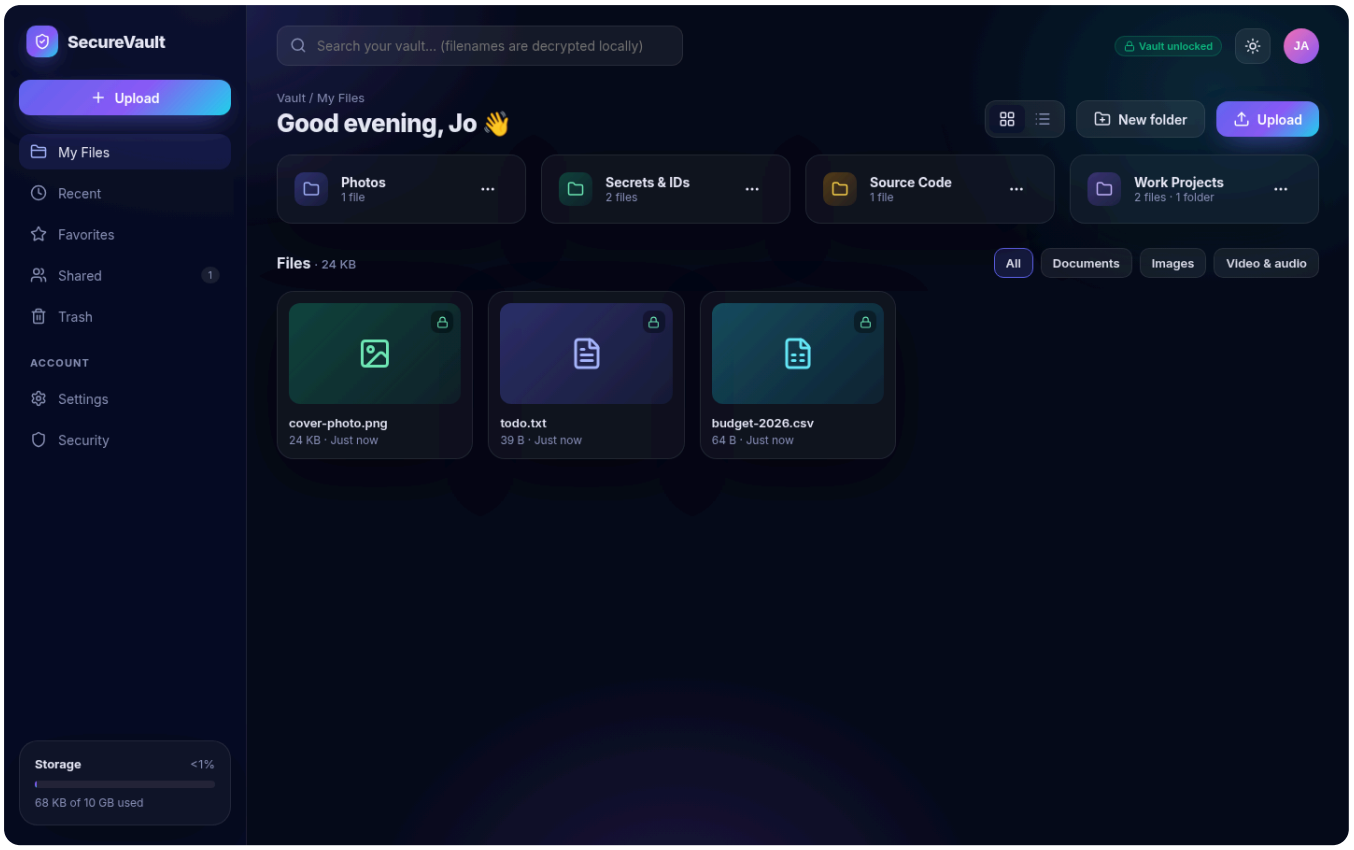
Contents

1. README.md
2. docs/ARCHITECTURE.md
3. docs/SECURITY.md
4. docs/API.md
5. docs/DATABASE.md
6. docs/COMPONENTS.md
7. docs/DEVELOPMENT.md
8. docs/DEPLOYMENT.md
9. docs/USER-GUIDE.md

🔐 SecureVault

Zero-knowledge encrypted file vault — upload any file, open it from any computer, and nobody but you can read it.

SecureVault is a full-stack **MERN** app (MongoDB + Express + React + Node) with **Prisma** as the ORM. Every file *and* filename is encrypted **in your browser** with AES-256-GCM before upload. The server only ever stores ciphertext, so even a full server/database breach doesn't expose your files.

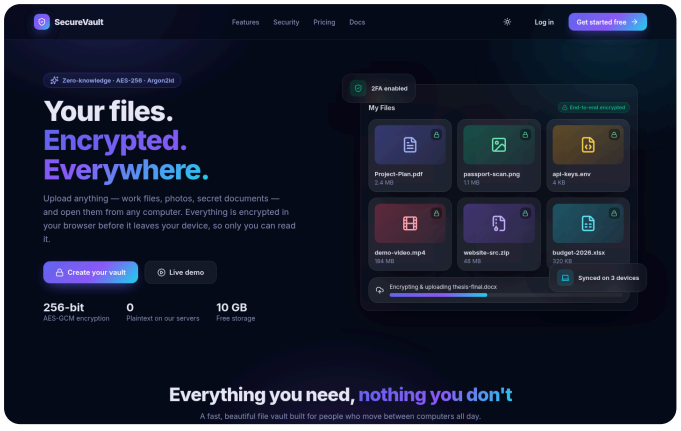


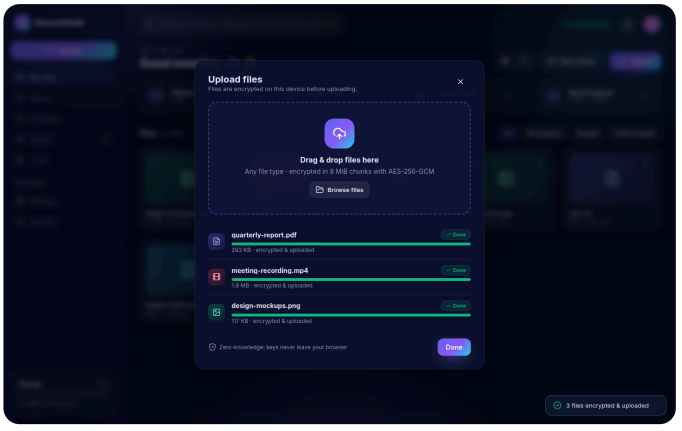
✨ Features

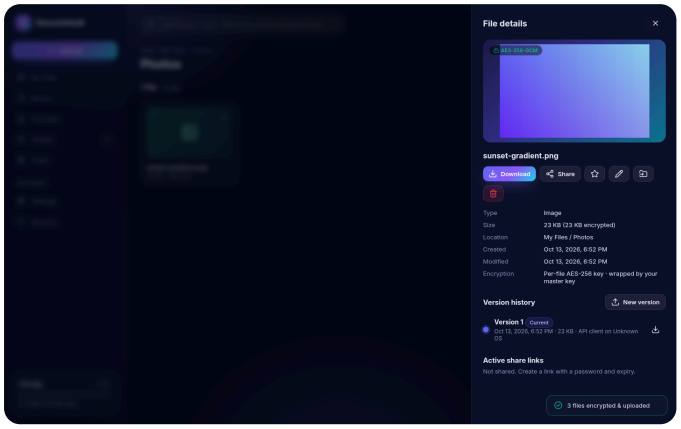
Area	What you get
Zero-knowledge crypto	Argon2id key derivation (PBKDF2 fallback), separate auth key, wrapped master key, per-file AES-256-GCM keys, encrypted filenames, 8 MiB chunked encryption with chunk-order protection, recovery key
Accounts	Sign-up with strength meter, recovery-key screen, login, TOTP 2FA (QR + backup codes), unlock-after-reload, password change (re-wraps keys), account recovery, delete account
Sessions	15-min JWT access tokens (memory only) + rotating httpOnly SameSite=Strict refresh cookie with reuse detection, device list + remote sign-out, login alerts in activity log
File manager	Folders (create / rename / move / nested / trash), drag & drop upload anywhere, multi-file upload with per-file progress, grid & list views, client-side search over decrypted names, favorites, recent, trash with restore / delete forever / empty, rename, move (dialog or drag onto folder)
Versions	Upload a new version, list, download any version, restore
Previews	Decrypted in the browser: images, PDF, video, audio, text/code

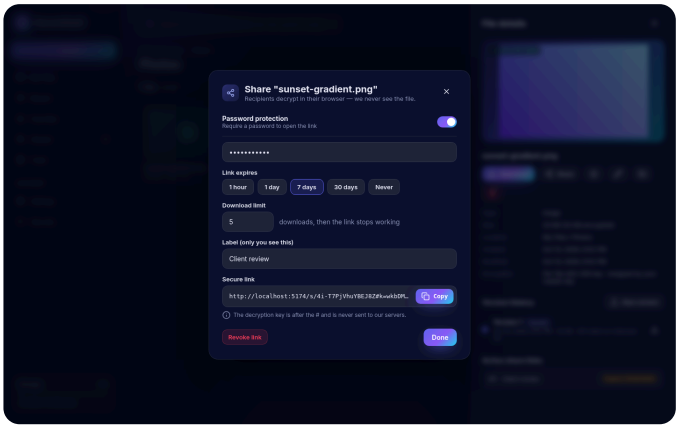
Area	What you get
Sharing	Links with the key in the URL `#fragment`, optional password (Argon2id), expiry, download limit, revoke, "Shared" view; public page decrypts in the recipient's browser
Storage	Pluggable: local disk (default) or S3 / Cloudflare R2 / Backblaze via pre-signed URLs; per-user quota + usage meter
Security	helmet + CSP, strict CORS, rate limiting, account lockout, NoSQL-injection sanitizer, zod validation everywhere, CSRF guard on cookie routes, body-size limits, ownership checks on every route, pino logging, no secrets in repo
UI	Dark navy/indigo glass design with gradient accents & lock badges, light/dark toggle, responsive (mobile sidebar), Framer Motion animations

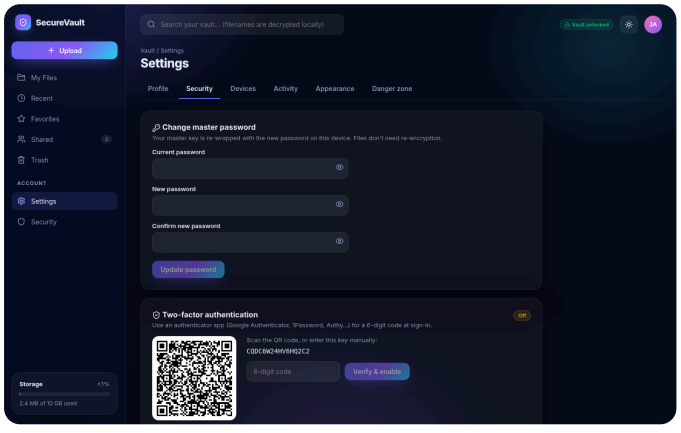
📸 Screenshots

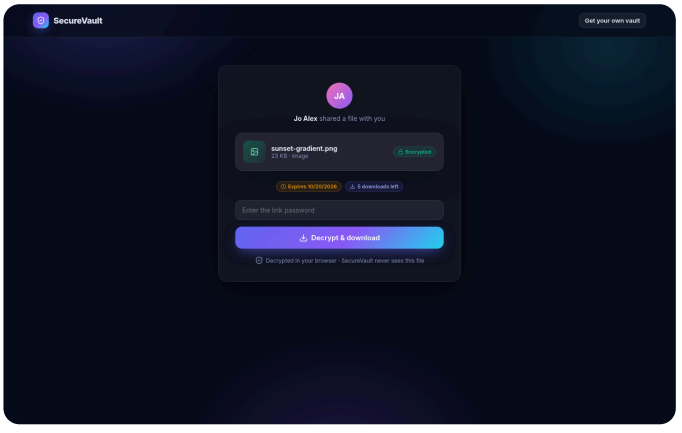


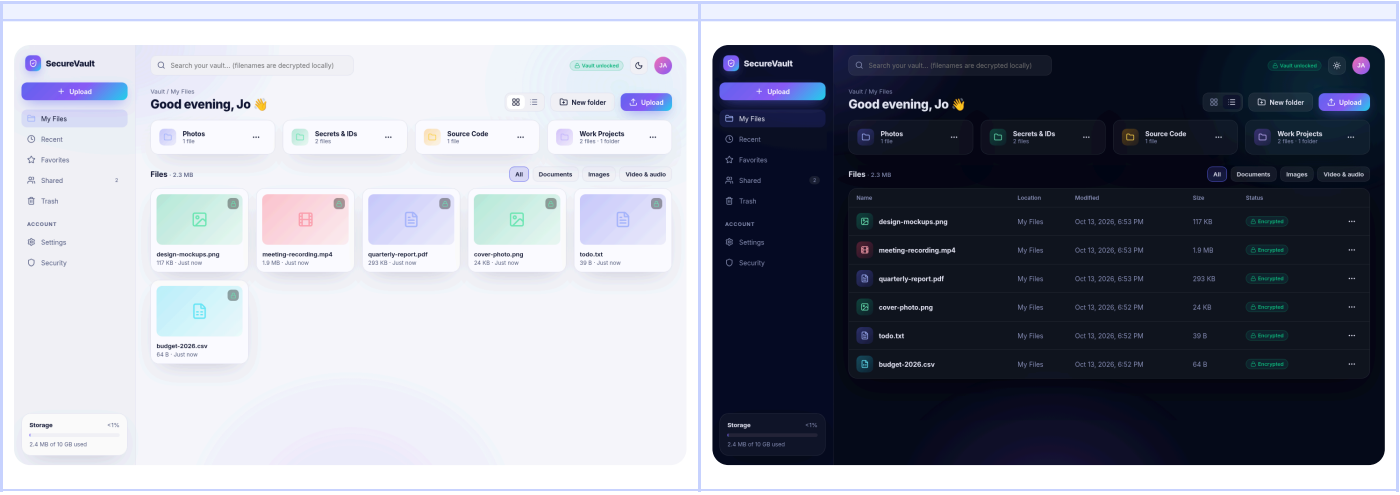












🚀 Quick start

Requirements: **Node.js 20+**. No database needed for local dev (an in-memory MongoDB replica set starts automatically).

```
# 1. API (port 5100) — seeds the demo account on start
cd server
npm install            # also runs `prisma generate`
cp .env.example .env   # optional; defaults work for local dev
SEED_DEMO=1 npm start

# 2. Web app (port 5174) — in a second terminal
cd client
npm install
npm run dev
```

Open **http://localhost:5174** and click **Live demo**, or log in with:

- **Email:** `demo@securevault.app`
- **Password:** `Demo@12345`

> ⚠️ Without `DATABASE_URL` the data lives in memory and disappears when the API stops. Point `DATABASE_URL` at MongoDB Atlas to keep it (see docs/DEPLOYMENT.md).

🧪 Tests

```
cd server && npm test          # Jest + Supertest: 69 tests (auth, 2FA, files, folders,
versions, trash, shares, quotas, authorization)
cd client && npm run e2e       # Playwright: real browser register → upload →
download/decrypt → share → public download
cd client && npx vite build    # production build
```

📚 Documentation

Doc	Contents
ARCHITECTURE.md	System design, data flows, folder structure
SECURITY.md	Threat model, crypto details, OWASP Top 10 mapping
API.md	Every endpoint with request/response
DATABASE.md	Prisma schema and model notes
COMPONENTS.md	Every React component, hook and store
DEVELOPMENT.md	Setup, scripts, Prisma, testing, linting
DEPLOYMENT.md	Atlas, R2/S3, Render/Railway, Vercel
USER-GUIDE.md	How to use SecureVault
SecureVault-Documentation.pdf	All of the above in one PDF

🗂 Project structure

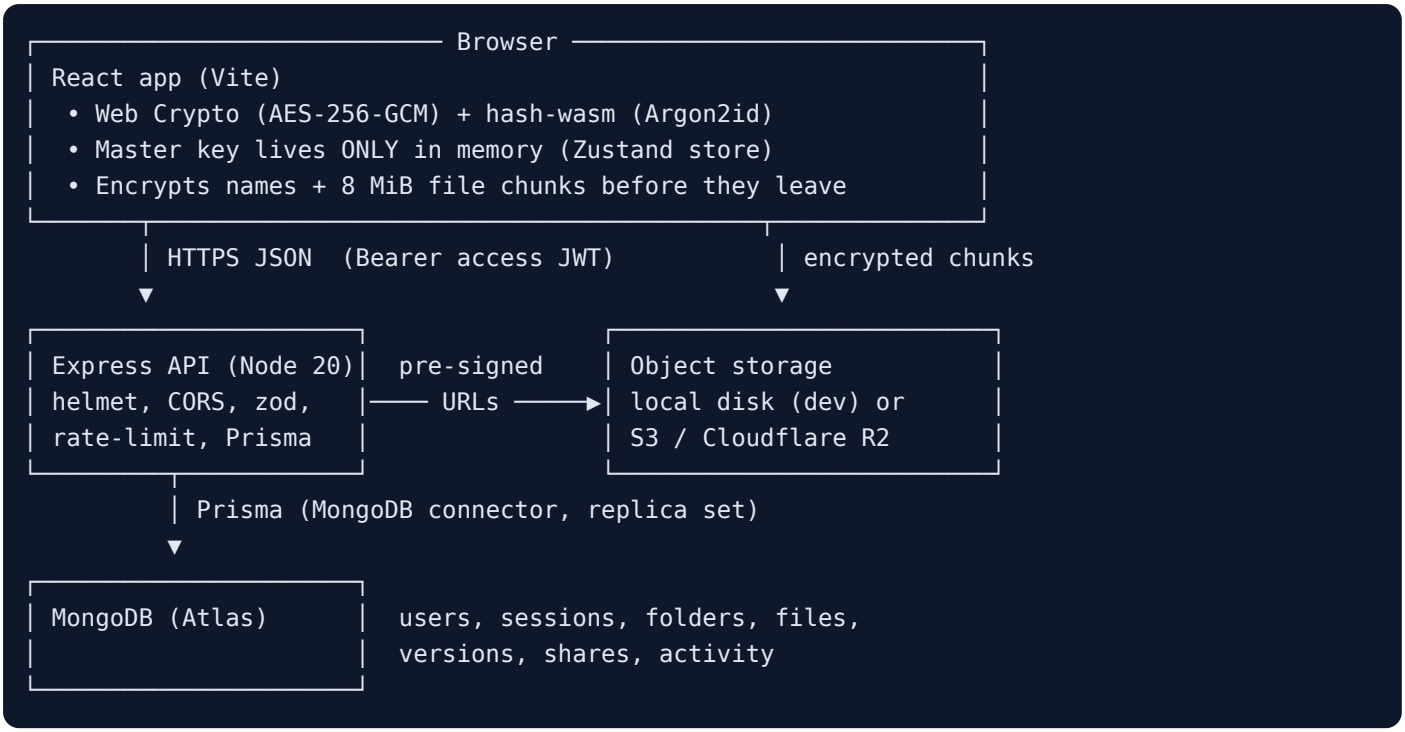
```
securevault/
├── client/                 React 18 + Vite + Tailwind + TanStack Query + Zustand
│   ├── src/crypto/         core.js (primitives) · vault.js (upload/download/share flows)
│   ├── src/api/            fetch wrapper with token refresh · account flows
│   ├── src/stores/         auth (keys in memory) · ui · theme
│   ├── src/hooks/          useVault · useUploader · useFileActions · useDecryptedFile
│   ├── src/components/     AppShell, Sidebar, FileCard, DetailsPanel, ShareDialog, …
│   ├── src/pages/          Landing, Signup, Login, Files, Shared, Settings, PublicShare, …
│   └── tests/e2e/          e2e.mjs · screenshots.mjs
├── server/                 Express (ESM) + Prisma (MongoDB)
│   ├── prisma/schema.prisma
│   ├── src/routes/         auth · account · files · shares
│   ├── src/middleware/     auth · validate · security · error
│   ├── src/services/       storage (local / s3) · tokens · vault · activity
│   ├── src/seed/           demo account seeder
│   └── tests/              Jest + Supertest
└── docs/                   documentation + screenshots + PDF
```

License

MIT — built by Jo Alex as a learning project. Review SECURITY.md before storing truly sensitive data.

Architecture

1. Big picture



- **Client** (`client/`): UI, key derivation, encryption/decryption, chunked upload/download.
- **API** (`server/`): authentication, sessions, metadata (encrypted), quotas, share-link rules, audit log. It **never** receives a plaintext file, filename or key.
- **Database**: MongoDB via Prisma. Prisma's MongoDB connector requires a replica set; Atlas clusters are replica sets. For local dev, `mongodb-memory-server` 's `MongoMemoryReplSet` is started automatically and `prisma db push` creates the indexes.
- **Storage driver** (`server/src/services/storage`): `local` stores each encrypted chunk at `storage/<storageKey>/<n>` and proxies bytes through authenticated API routes. `s3` returns 5-minute pre-signed PUT/GET URLs so the browser talks to S3/R2 directly.

2. Key flows

Sign-up

1. Browser generates a random 16-byte salt and runs **Argon2id(password)** → 64 bytes → `KEK` (32) + `authKey` (32).
2. Generates random **master key (MK)** and **recovery key (RK)**.
3. Sends `authKey` , salt, KDF params, `AES-GCM(KEK, MK)` , `AES-GCM(SHA256(RK), MK)` and a recovery proof. The server Argon2id-hashes the auth key and recovery proof again.
4. The recovery key is shown once; the session is only activated after the user confirms they saved it.

Login

`POST /auth/prelogin` (salt) → derive keys locally → `POST /auth/login` (authKey) → optional `POST /auth/2fa/verify` → access token + refresh cookie → unwrap MK with KEK in the browser.

On page reload the refresh cookie restores the *session* but not the *master key*, so the user sees **Unlock vault** and re-enters the password (no 2FA needed). Keys are never persisted to storage.

Upload (per file, per version)

1. Random `FK` (file key) + 8-byte nonce prefix.
2. `POST /files` with encrypted name/mime, size, chunk count, `AES-GCM(MK, FK)` → server creates `File` + `Version` and returns part URLs.
3. For each 8 MiB chunk *i*: `AES-GCM(FK, iv = prefix‖i, AAD = "sv-chunk:i:count")` → `PUT` part.
4. `POST /uploads/:vid/complete` → server checks all parts arrived, quota, marks ready, charges storage.

Download / preview

`GET /files/:id/download` returns the version's wrapped key and part URLs → fetch → decrypt each chunk → `Blob` → object URL for preview or save.

Sharing

Browser creates random link key `K`; uploads `FK` wrapped with `K` (or with `SHA256(K‖Argon2id(password))`) plus name/mime encrypted with `K`. The link is `/s/<token>#k=<K>`; fragments are never sent to servers. The public page reads `K` from `location.hash`, fetches metadata, verifies password proof server-side, gets a 10-minute download ticket, and decrypts locally.

3. Folder structure

```
server/src
├── app.js                Express app + middleware order
├── index.js              boot: connect DB, listen, optional seed
├── config/env.js         env vars (+ safe dev fallbacks)
├── config/db.js          Prisma client, in-memory replica set fallback
├── middleware/           auth (JWT + live session check), validate (zod), security
(sanitize, CSRF), error
├── routes/               auth.js · account.js · files.js · shares.js
├── services/             tokens (JWT/refresh rotation) · storage/{local,s3} · vault
(ownership/purge) · activity
├── seed/                 demo.js (seed via API using client crypto) · nodeHttp.js
└── utils/                logger (pino) · hash (Argon2id) · crypto (server secrets) · errors

client/src
├── crypto/core.js        primitives: KDF, AES-GCM, chunk crypto, recovery key, share keys
├── crypto/vault.js       flows: uploadFile, downloadFile, createShare, downloadShare
├── api/http.js           fetch wrapper (+ silent refresh) & http adapter
├── api/auth.js           register, login, 2FA, unlock, password change, recovery
├── stores/               Zustand: auth (in-memory keys), ui (toasts/uploads/dialogs), theme
├── hooks/                TanStack Query hooks, uploader, actions, decrypted file
```

```
├── components/            layout, file UI, dialogs, ui/ primitives
└── pages/                 route screens
```

4. Why these choices

Choice	Reason
Encrypt in the browser	Zero-knowledge: the server can't leak what it never had
Argon2id (hash-wasm)	Memory-hard KDF, OWASP recommended; WASM works in browser *and* Node (shared code)
Per-file keys	Sharing one file never exposes others; key rotation per version
Chunked AES-GCM	Large files without loading everything into memory at once; per-chunk integrity
Access token in memory + httpOnly refresh cookie	XSS can't steal a long-lived token; CSRF blocked by SameSite=Strict + Origin + custom header
Prisma	Typed schema, clear models for learners, easy to switch providers
Pluggable storage	Local disk for zero-setup dev, S3/R2 for production

Security

1. Threat model

Asset	Protected against	How
File contents & names	Server/database/storage breach, curious operator, storage provider	Encrypted in browser (AES-256-GCM); server stores ciphertext only
Master key	Server breach, network attacker	Never leaves browser unencrypted; wrapped with password-derived KEK
Password	Server breach, offline cracking	Never sent; Argon2id (64 MiB, t=3) in browser → auth key, which the server hashes again with Argon2id
Session	XSS token theft, CSRF, stolen refresh token	Access JWT in memory (15 min); refresh token httpOnly + SameSite=Strict, rotated each use, reuse ⇒ revoke
Account	Brute force, credential stuffing	Rate limit per IP+email, lockout after 10 failures, TOTP 2FA, backup codes
Share links	Link interception (server side), guessing	Key in URL fragment (never sent to server); 96-bit random token; optional password; expiry + download limit
Other users' data	Broken access control	Every query filters by `ownerId`; non-owned ids return 404

Out of scope / limitations: a compromised browser or device (malware, malicious extension) can read decrypted data; a malicious server could serve modified JavaScript (mitigate with SRI/reproducible builds/desktop app); metadata visible to the server includes file sizes, timestamps, folder structure and counts; whoever has a full share URL (incl. fragment) and password can download.

2. Cryptography details

Item	Algorithm	Parameters
Password KDF	Argon2id (hash-wasm)	m = 64 MiB, t = 3, p = 1, 16-byte random salt, 64-byte output. Fallback: PBKDF2-SHA256, 600 000 iterations
KEK / authKey	split of KDF output	bytes 0-31 = KEK, 32-63 = authKey (hex)
Server password storage	Argon2id	m = 19 MiB, t = 2 (OWASP minimum), random salt — hashes the authKey
Master key (MK)	random 256-bit	`encMasterKey = AES-GCM(KEK, MK, AAD "sv-key")`
Recovery key (RK)	random 256-bit, shown as 52 base32 chars	wrap key = SHA-256("sv-rk-enc"‖RK); proof = SHA-256("sv-rk-auth"‖RK) (stored Argon2id-hashed)
File key (FK)	random 256-bit per **version**	`encFileKey = AES-GCM(MK, FK, AAD "sv-key")`
Names / MIME	AES-256-GCM with MK	random 96-bit IV, AAD "sv-text"
File chunks	AES-256-GCM with FK	8 MiB plaintext; IV = 8-byte random prefix ‖ uint32 chunk index; AAD = `sv-chunk:<index>:<count>` (prevents reordering, truncation, cross-file swapping)

Item	Algorithm	Parameters
Share key	random 256-bit link key K in `#k=`	wrap key = K, or SHA-256(K ‖ Argon2id(password, salt; m=19 MiB, t=2)); password proof = second half of that Argon2id output (server stores Argon2id(proof))
TOTP secret	AES-256-GCM with server key	`SERVER_ENC_KEY` env var; backup codes stored as SHA-256
Refresh token	256-bit random	stored as SHA-256

All AES-GCM blobs are `base64(iv || ciphertext || tag)`. GCM's tag makes any tampering fail decryption.

Password change re-derives KEK and re-wraps MK only. **Recovery** decrypts MK with RK in the browser, then re-wraps it with a new password, and signs out all sessions.

3. Application security controls

Control	Implementation
Security headers	`helmet` — CSP `default-src 'none'; frame-ancestors 'none'` on the API, nosniff, Referrer-Policy, HSTS in production, `x-powered-by` disabled
Front-end CSP	No inline scripts (theme bootstrap is an external file); set CSP at your static host — see DEPLOYMENT.md
CORS	Allow-list from `CLIENT_ORIGIN`, credentials enabled only for it
Rate limiting	`express-rate-limit`: global 1200/min/IP; auth routes 20 / 15 min per IP+email; public share routes 60 / 15 min
Lockout	10 failed logins → 15-minute lock (423)
Input validation	`zod` schemas on every body/params/query; ObjectId regex; base64 regex; size bounds
NoSQL injection	sanitizer strips `$`-prefixed / dotted keys (equivalent to express-mongo-sanitize) + zod + Prisma typed queries
CSRF	Cookie only sent to `/api/v1/auth` with SameSite=Strict; `/refresh` and `/logout` additionally require `X-Requested-With: securevault` and a matching `Origin`
Size limits	JSON 100 KB; chunk uploads ≤ 8 MiB + 1 KiB; chunk count must match size
Access control	`ownFile/ownFolder/ownVersion` helpers; every list/update filters by `ownerId`; foreign ids → 404
Sessions	Session row checked on every request → device revoke is immediate
Account enumeration	`/prelogin` returns a deterministic fake salt for unknown emails; generic login errors
Logging	`pino` structured logs with redaction of auth headers & cookies; `Activity` audit log (login, failed login, 2FA, uploads, shares, device revokes…)
Secrets	Only via env vars; `.env.example` contains no secrets; dev generates random secrets per boot
Previews	Decrypted blobs rendered via object URLs; PDFs in a sandboxed iframe

4. OWASP Top 10 (2021) mapping

Risk	Mitigation in SecureVault
A01 Broken Access Control	Ownership filter on every route; tested with a two-user test matrix (`files.test.js`, `share.test.js`)

Risk	Mitigation in SecureVault
A02 Cryptographic Failures	Client-side AES-256-GCM, Argon2id, per-file keys, TLS in production, encrypted TOTP secrets, hashed refresh tokens
A03 Injection	zod validation, sanitizer, Prisma parameterised queries, React escaping
A04 Insecure Design	Zero-knowledge architecture, threat model above, recovery-key design
A05 Security Misconfiguration	helmet, strict CORS, generic 500 messages (no stack traces), env-based config
A06 Vulnerable Components	Lockfiles; run `npm audit` in CI (see DEVELOPMENT.md)
A07 Identification & Auth Failures	Rate limits, lockout, 2FA + backup codes, rotating refresh tokens with reuse detection, device management
A08 Software & Data Integrity	GCM auth tags on every blob/chunk, chunk AAD binding, lockfiles
A09 Logging & Monitoring	pino logs + per-user activity log with sign-in alerts
A10 SSRF	API never fetches user-supplied URLs; storage keys are generated server-side

5. Roadmap

- **Passkeys (WebAuthn)** via `@simplewebauthn/server` + `/browser` — second factor and passwordless unlock (PRF extension for key wrapping).
- Email alerts for new-device sign-ins (Resend/Postmark) — today they appear in the activity log.
- Redis-backed rate limiting for multi-instance deployments.
- Subresource Integrity and a signed desktop app to reduce "malicious JS" risk.
- Public-key sharing between SecureVault users (X25519).

6. Reporting

Please report vulnerabilities privately to the maintainer rather than opening a public issue.

REST API

Base URL: `/api/v1` · JSON unless noted · Auth: `Authorization: Bearer <accessToken>` (🔒) unless marked **public**.

Errors always look like:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "email: Invalid email" } }
```

Common codes: `VALIDATION_ERROR` 400, `BAD_REQUEST` 400, `UNAUTHORIZED` 401, `INVALID_CREDENTIALS` 401, `FORBIDDEN` 403, `NOT_FOUND` 404, `EMAIL_TAKEN` 409, `INCOMPLETE_UPLOAD` 409, `LINK_EXPIRED` / `LINK_LIMIT` / `LINK_REVOKED` 410, `QUOTA_EXCEEDED` / `PAYLOAD_TOO_LARGE` 413, `LOCKED` 423, `RATE_LIMITED` 429.

Types: `b64` = base64 string of AES-GCM blob (`iv‖ct‖tag`), `hex64` = 64 hex chars, `id` = 24-char ObjectId.

Health

`GET /health` — public

`200 { "ok": true, "storage": "local" }`

Auth (`/auth`)

`POST /auth/prelogin` — public, rate-limited

Body `{ email }` → `200 { kdfSalt, kdfParams: { alg, m, t, p } }` (fake but stable salt for unknown emails).

`POST /auth/register` — public, rate-limited

```
{ "email": "jo@example.com", "name": "Jo Alex", "authKey": "hex64", "kdfSalt": "b64",
  "kdfParams": { "alg": "argon2id", "m": 65536, "t": 3, "p": 1 },
  "encMasterKey": "b64", "encMasterKeyRecovery": "b64", "recoveryProof": "hex64" }
```

→ `201 { accessToken, user }` + `Set-Cookie: sv_rt=…; HttpOnly; SameSite=Strict; Path=/api/v1/auth`. `409 EMAIL_TAKEN`.

`user` object (also returned by `/auth/me`):

```
{ "id": "…", "email": "…", "name": "…", "kdfSalt": "…", "kdfParams": {…}, "encMasterKey":
"b64",
  "totpEnabled": false, "storageUsed": 0, "storageQuota": 10737418240, "createdAt": "…" }
```

`POST /auth/login` — public, rate-limited

Body `{ email, authKey }` → `200 { accessToken, user }` + cookie, **or** `200 { twoFactorRequired: true, ticket }`. `401 INVALID_CREDENTIALS`, `423 LOCKED`.

`POST /auth/2fa/verify` — public, rate-limited

Body `{ ticket, code }` (6-digit TOTP or backup code) → `200 { accessToken, user }` + cookie. `401 INVALID_CODE`.

`POST /auth/refresh` — cookie, CSRF-guarded

Headers `X-Requested-With: securevault` (+ allowed `Origin`). → `200 { accessToken, user }` with rotated cookie; `204` if no cookie; `401` if expired/revoked/reused; `403` CSRF failure.

`POST /auth/logout` — cookie, CSRF-guarded

→ `200 { ok: true }`, revokes the session and clears the cookie.

`POST /auth/recover/start` — public

Body `{ email, recoveryProof }` → `200 { encMasterKeyRecovery }`. `401 INVALID_RECOVERY`.

`POST /auth/recover` — public

Body `{ email, recoveryProof, authKey, kdfSalt, kdfParams, encMasterKey }` → `200 { ok: true }`; all sessions revoked.

`GET /auth/me` 🔒 → `200 { user }`

`PATCH /auth/me` 🔒 — Body `{ name }` → `200 { user }`

`DELETE /auth/me` 🔒 — Body `{ authKey }` → `200 { ok: true }` (deletes all data). `401` wrong password.

`POST /auth/password` 🔒

Body `{ currentAuthKey, authKey, kdfSalt, kdfParams, encMasterKey }` → `200 { user }`; other sessions revoked.

`POST /auth/2fa/setup` 🔒 → `200 { secret, otpauth, qr }` (`qr` = PNG data URL)

`POST /auth/2fa/enable` 🔒 — Body `{ code }` → `200 { backupCodes: string[10] }`

`POST /auth/2fa/disable` 🔒 — Body `{ code }` (TOTP or backup) → `200 { ok: true }`

Devices & activity

`GET /devices` 🔒

`200 { devices: [{ id, name, ip, lastSeen, createdAt, current }] }`

`DELETE /devices/:id` 🔒 → `200 { ok: true }` (that device's tokens stop working immediately)

`GET /activity?page=1` 🔒

`200 { page, items: [{ id, action, detail, ip, ua, createdAt }] }` (30 per page, newest first). Actions include `login`, `login.failed`, `2fa.*`, `file.upload`, `file.version`, `file.trash`, `file.restore`, `file.delete`, `file.rename`, `file.move`, `folder.*`, `share.create`, `share.revoke`, `password.changed`, `password.recovered`, `device.revoked`, `trash.empty`.

Files

`GET /files?view=folder|all|recent|favorites|trash&folder=<id>` 🔒

```
{ "files": [{ "id", "folderId", "encName", "encMime", "size", "favorite", "trashed",
"trashedAt",
              "versionCount", "currentVersionId", "createdAt", "updatedAt", "lastOpenedAt"
}],
  "folders": [{ "id", "parentId", "encName", "trashed", "trashedAt", "createdAt",
"updatedAt", "fileCount" }],
  "storageUsed": 123, "storageQuota": 10737418240 }
```

`folder` view lists direct children of `folder` (root when omitted). `all` returns every live file/folder (used for client-side search). Items inside trashed folders are hidden outside the trash.

`POST /files` 🔒 — start upload of a new file

```
{ "folderId": null, "encName": "b64", "encMime": "b64", "size": 10485883,
  "chunkSize": 8388608, "chunkCount": 2, "encFileKey": "b64", "noncePrefix": "12-char b64" }
```

→ `201 { file, version: { id, number, size, chunkSize, chunkCount, encFileKey, noncePrefix, status, device, createdAt }, parts: [{ n, url, direct }] }`. `direct: false` → `url` is an API path (`/api/v1/uploads/:vid/parts/:n`, needs the Bearer token); `direct: true` → pre-signed S3/R2 URL. `413 QUOTA_EXCEEDED`; `400` if `chunkCount ≠ ceil(size/chunkSize)`.

`POST /files/:id/versions` 🔒 — start a new version

Body `{ size, chunkSize, chunkCount, encFileKey, noncePrefix }` → `201 { version, parts }`.

`PUT /uploads/:vid/parts/:n` 🔒 — local driver

Body: raw `application/octet-stream` encrypted chunk (16 B … chunkSize+16 B). → `200 { ok, n }`. `413` if too large.

`POST /uploads/:vid/parts/:n/done` 🔒 — S3 driver

Marks a directly-uploaded part as received → `200 { ok }`.

`POST /uploads/:vid/complete` 🔒

→ `200 { file }`. `409 INCOMPLETE_UPLOAD` if a part is missing.

`GET /files/:id/download?version=<vid>` 🔒

→ `200 { version, parts: [{ n, url, direct }] }` (current version when omitted).

`GET /uploads/:vid/parts/:n` 🔒 — local driver

→ `200` raw encrypted chunk.

`PATCH /files/:id` 🔒

Body any of `{ encName, folderId (id|null), favorite }` → `200 { file }`.

`DELETE /files/:id` 🔒 → move to trash. `?permanent=true` → erase file, versions, chunks, shares; frees quota.

`POST /files/:id/restore` 🔒 → `200 { file }` (to root if its folder is gone)

`DELETE /trash` 🔒 → `200 { ok, freed }` (permanently empties trash, incl. trashed folders' contents)

`GET /files/:id/versions` 🔒

→ `200 { currentVersionId, versions: [version…] }` newest first.

`POST /files/:id/versions/:vid/restore` 🔒 → `200 { file }`

Folders

`GET /folders` 🔒 → `200 { folders }` (all live folders; for breadcrumbs & move dialog)

`POST /folders` 🔒 — Body `{ encName, parentId? }` → `201 { folder }`

`PATCH /folders/:id` 🔒 — Body `{ encName?, parentId? }` → `200 { folder }`; `400` when moving into itself/descendant

`DELETE /folders/:id` 🔒 — trash; `?permanent=true` erases the folder tree and its files

`POST /folders/:id/restore` 🔒 → `200 { folder }`

Shares (owner)

`POST /shares` 🔒

```
{ "fileId": "id", "encName": "b64", "encMime": "b64", "encFileKey": "b64", "label": "Client
review",
  "passSalt": "b64?", "passProof": "hex64?", "expiresAt": "ISO date | null", "maxDownloads":
5 }
```

→ `201 { share: { id, token, fileId, label, hasPassword, expiresAt, maxDownloads, downloads, status, createdAt } }`. The link key is **not** sent; the client builds `/s/<token>#k=<key>`.

`GET /shares` 🔒 → `200 { shares: [share + fileEncName, fileEncMime, fileTrashed] }`; `status` ∈ `active|expired|limit|revoked`

`DELETE /shares/:id` 🔒 → revoke → `200 { ok: true }`

Shares (public)

`GET /public/s/:token` — public, rate-limited

→ `200 { encName, encMime, size, ownerName, hasPassword, passSalt, expiresAt, downloadsLeft }` · `404` · `410 LINK_*`.

`POST /public/s/:token/download` — public, rate-limited

Body `{ passProof? }` → increments the counter and returns `200 { encFileKey, noncePrefix, chunkSize, chunkCount, size, ticket, parts }`. `401 WRONG_PASSWORD`.

`GET /public/s/:token/parts/:n?t=<ticket>` — public

→ raw encrypted chunk (local driver). Ticket = 10-minute JWT bound to the version. `401 TICKET_EXPIRED`.

Database

SecureVault uses **MongoDB** through **Prisma ORM** (`provider = "mongodb"`).

- Prisma's MongoDB connector **requires a replica set** (it uses transactions). MongoDB Atlas clusters are replica sets out of the box.
- For local development, if `DATABASE_URL` is empty the API starts `MongoMemoryReplSet` (mongodb-memory-server), sets `DATABASE_URL` to it **before** creating the `PrismaClient`, and runs `prisma db push` to create the unique indexes (`server/src/config/db.js`).
- Fields named `enc*` contain ciphertext produced in the browser: the server can't read them.

Prisma schema (`server/prisma/schema.prisma`)

```
// SecureVault database schema (Prisma + MongoDB).
// Fields named enc* hold ciphertext produced in the browser (base64 of iv||ciphertext).
// The server can never read them — this is the zero-knowledge design.

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

type KdfParams {
  alg  String
  m    Int?
  t    Int?
  p    Int?
  iter Int?
}

type Totp {
  enabled       Boolean  @default(false)
  encSecret     String?  // TOTP secret sealed with SERVER_ENC_KEY
  pendingSecret String?
  backupCodes   String[] // sha256 hashes, single use
}

model User {
  id                  String     @id @default(auto()) @map("_id") @db.ObjectId
  email               String     @unique
  name                String     @default("")
  authHash            String     // argon2id(clientAuthKey)
  kdfSalt             String
  kdfParams           KdfParams
  encMasterKey        String     // AES-GCM(KEK, MK)
  encMasterKeyRecovery String     // AES-GCM(RecoveryKey, MK)
```

```
  recoveryHash         String     // argon2id(recoveryProof)
  totp                 Totp?
  storageUsed          Float      @default(0)
  storageQuota         Float
  failedLogins         Int        @default(0)
  lockedUntil          DateTime?
  createdAt            DateTime   @default(now())
  updatedAt            DateTime   @updatedAt
  sessions             Session[]
  folders              Folder[]
  files                File[]
  versions             Version[]
  shares               Share[]
  activities           Activity[]
}

/// A signed-in device = one rotating refresh-token session.
model Session {
  id          String   @id @default(auto()) @map("_id") @db.ObjectId
  userId      String   @db.ObjectId
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  refreshHash String
  name        String
  ua          String
  ip          String
  lastSeen    DateTime @default(now())
  expiresAt   DateTime
  revoked     Boolean  @default(false)
  createdAt   DateTime @default(now())

  @@index([userId])
}

model Folder {
  id        String    @id @default(auto()) @map("_id") @db.ObjectId
  ownerId   String    @db.ObjectId
  owner     User      @relation(fields: [ownerId], references: [id], onDelete: Cascade)
  parentId  String?   @db.ObjectId
  encName   String
  trashed   Boolean   @default(false)
  trashedAt DateTime?
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt

  @@index([ownerId, parentId])
}

model File {
  id               String    @id @default(auto()) @map("_id") @db.ObjectId
  ownerId          String    @db.ObjectId
  owner            User      @relation(fields: [ownerId], references: [id], onDelete:
Cascade)
  folderId         String?   @db.ObjectId
  encName          String
  encMime          String
```

```
  size             Float     @default(0) // plaintext size of current version
  status           String    @default("uploading") // uploading | ready
  currentVersionId String?   @db.ObjectId
  versionCount     Int       @default(0)
  favorite         Boolean   @default(false)
  trashed          Boolean   @default(false)
  trashedAt        DateTime?
  lastOpenedAt     DateTime?
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt
  versions         Version[]
  shares           Share[]

  @@index([ownerId, folderId])
}

/// Every upload (first or later) is a Version with its own file key and chunks.
model Version {
  id          String   @id @default(auto()) @map("_id") @db.ObjectId
  fileId      String   @db.ObjectId
  file        File     @relation(fields: [fileId], references: [id], onDelete: Cascade)
  ownerId     String   @db.ObjectId
  owner       User     @relation(fields: [ownerId], references: [id], onDelete: Cascade)
  number      Int
  size        Float    // plaintext bytes
  storedBytes Float    @default(0)
  chunkSize   Int
  chunkCount  Int
  encFileKey  String   // AES-GCM(MK, FK)
  noncePrefix String   // 8 random bytes, base64
  storageKey  String
  partsDone   Int[]
  status      String   @default("uploading")
  device      String?
  createdAt   DateTime @default(now())
  shares      Share[]

  @@index([fileId])
}

model Share {
  id           String    @id @default(auto()) @map("_id") @db.ObjectId
  ownerId      String    @db.ObjectId
  owner        User      @relation(fields: [ownerId], references: [id], onDelete: Cascade)
  fileId       String    @db.ObjectId
  file         File      @relation(fields: [fileId], references: [id], onDelete: Cascade)
  versionId    String    @db.ObjectId
  version      Version   @relation(fields: [versionId], references: [id], onDelete: Cascade)
  token        String    @unique
  label        String?
  encName      String    // filename encrypted with the link key (#fragment)
  encMime      String
  encFileKey   String    // FK wrapped with link key (+ password key if set)
  passSalt     String?
  passHash     String?
```

```
  expiresAt    DateTime?
  maxDownloads Int?
  downloads    Int       @default(0)
  revoked      Boolean   @default(false)
  createdAt    DateTime  @default(now())

  @@index([ownerId])
}

model Activity {
  id        String   @id @default(auto()) @map("_id") @db.ObjectId
  userId    String   @db.ObjectId
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  action    String
  detail    String   @default("")
  ip        String   @default("")
  ua        String   @default("")
  createdAt DateTime @default(now())

  @@index([userId, createdAt])
}
```

Models at a glance

Model	Purpose	Notable fields
`User`	Account + key material	`authHash` (Argon2id of client auth key), `kdfSalt`, `kdfParams`, `encMasterKey`, `encMasterKeyRecovery`, `recoveryHash`, `totp` (embedded type), `storageUsed`, `storageQuota`, lockout counters
`Session`	One signed-in device	`refreshHash` (SHA-256), `name` (e.g. "Chrome on Windows"), `ip`, `lastSeen`, `expiresAt`, `revoked`
`Folder`	Tree of folders	`parentId` (null = root), `encName`, `trashed`
`File`	A file entry	`folderId`, `encName`, `encMime`, `size`, `currentVersionId`, `favorite`, `trashed`, `status`
`Version`	One uploaded version	`number`, `size`, `chunkSize`, `chunkCount`, `encFileKey`, `noncePrefix`, `storageKey`, `partsDone`, `status`
`Share`	Share link	`token` (unique), `versionId`, `encName` / `encMime` / `encFileKey` (wrapped with link key), `passSalt`, `passHash`, `expiresAt`, `maxDownloads`, `downloads`, `revoked`
`Activity`	Audit log	`action`, `detail`, `ip`, `ua`, `createdAt`

Relationships

```
User 1—* Session
User 1—* Folder (parentId → Folder)
User 1—* File   (folderId → Folder)
File 1—* Version
File 1—* Share *—1 Version
User 1—* Activity
```

Storage accounting

`User.storageUsed` = sum of plaintext sizes of all **ready** versions. It increases on `/uploads/:vid/complete` and decreases when files are permanently deleted (`purgeFiles`). Quota is checked when an upload starts and again on completion.

Indexes

- `User.email` unique, `Share.token` unique
- `Session(userId)` , `Folder(ownerId, parentId)` , `File(ownerId, folderId)` , `Version(fileId)` , `Share(ownerId)` , `Activity(userId, createdAt)`

Common commands

```
cd server
npx prisma generate                       # regenerate the client after editing schema.prisma
DATABASE_URL="mongodb+srv://..." npx prisma db push   # create/update indexes on Atlas
npx prisma studio                         # browse data (needs DATABASE_URL)
```

Front-end components

The client is React 18 + Vite. Styling = the mockup design system (`src/design.css`, copied from the design package) plus small additions in `src/index.css`; Tailwind utilities are available (`preflight` disabled so the design reset wins). Icons: `lucide-react`; animations: `framer-motion`; server state: TanStack Query; client state: Zustand.

Pages (`src/pages`)

Component	Route	What it does
`Landing`	`/`	Marketing page: hero with animated mock vault, features, "how it stays private" steps, pricing, footer, theme toggle, "Live demo" (pre-fills demo login)
`Signup`	`/signup`	Name/email/password, `PasswordStrength` meter, acknowledgement switch; generates keys locally via `api/auth.register`
`RecoveryKey`	`/recovery-key`	Shows the 52-char recovery key in 13 groups, download/copy, confirmation switch; activates the pending session
`Login`	`/login`	Password step → optional 2FA step (`OtpInput` or backup code). If a session exists but the page was reloaded, shows **Unlock vault** (password only)
`Recover`	`/recover`	Email + recovery key + new password → re-wraps master key
`FilesPage`	`/app`, `/app/folder/:id`, `/app/recent`, `/app/favorites`, `/app/trash`, `/app/search`	Greeting/breadcrumbs, grid/list toggle, new folder, upload, folder cards, type filters, file grid/table, empty states, empty trash
`SharedPage`	`/app/shared`	Table of your share links with status, downloads, expiry, revoke
`SettingsPage`	`/app/settings?tab=`	Tabs: Profile, Security (change password, 2FA setup/disable, passkeys roadmap), Devices, Activity, Appearance, Danger zone (delete account)
`PublicShare`	`/s/:token#k=…`	Recipient page: reads key from fragment, shows sender & file, password field, decrypts & downloads in browser
`AuthLayout`	—	Two-column auth layout (story panel + form card)
`NotFound`	`*`	404 page

Layout & feature components (`src/components`)

Component	Purpose
`AppShell`	Logged-in layout: `Sidebar` + `Topbar` + routed page + `DialogHost`; drag files from the desktop anywhere to upload into the current folder (overlay)
`Sidebar`	Upload button, nav (My Files, Recent, Favorites, Shared with active-link count, Trash), Settings/Security, `StorageMeter`; slides in on mobile
`Topbar`	Search box (client-side over decrypted names → `/app/search`), "Vault unlocked" badge, theme toggle, account menu (lock & sign out)
`StorageMeter`	Used / quota with gradient bar (`GET /auth/me`)

Component	Purpose
`FolderCard`	Folder tile with file count; click to open; drop target for file cards; menu: rename, move, trash / restore, delete forever
`FileCard`	Grid tile: type-tinted thumbnail, lock badge, favorite star, size · time; draggable; opens details
`FileTable`	List view: name, location, modified, size, Encrypted/Shared badge, actions
`FileIcon`	Coloured icon square for a MIME type/extension (`lib/format.fileKind`)
`FileMenu`	"..." actions for a file (details, download, share, favorite, rename, move, trash; restore/delete in trash)
`UploadModal`	Drop zone + file picker; per-file progress (encrypting / chunk x of y / done / failed)
`DetailsPanel`	Slide-over: decrypted `Preview` , action buttons, metadata, `VersionHistory` , active share links
`Preview`	Renders decrypted blob: image, video, audio, PDF (sandboxed iframe), text/code; progress while decrypting
`VersionHistory`	Version timeline; upload new version; download or restore any version
`ShareDialog`	Password switch, expiry chips, download limit, label → creates link, copy, revoke
`MoveDialog`	Folder tree picker (blocks moving a folder into itself)
`DialogHost`	Renders the active dialog from `useDialogs` + details panel + upload modal

UI primitives (`src/components/ui`)

`Logo` , `Switch` (accessible toggle), `Modal` (animated glass dialog, Esc/backdrop to close), `Toasts` , `ThemeToggle` , `PasswordInput` (show/hide), `PasswordStrength` (4-segment meter + checks; scoring in `lib/password.js`), `OtpInput` (6 boxes, paste support), `Dropdown` , `EmptyState` , `PromptDialog` , `ConfirmDialog` .

Hooks (`src/hooks`)

Hook	Purpose
`useListing(view, folderId)`	Query `/files` and decrypt names with the master key
`useAllFolders()`	All folders, decrypted (breadcrumbs, move dialog)
`useShares()`	Share links with decrypted file names
`useRefresh()`	Invalidate file/folder/share/me queries after mutations
`useUploader()`	Upload queue: encrypt + upload files, progress into `useUI` , toasts
`useFileActions()`	One place for rename/move/favorite/trash/restore/delete/share/download/new folder/empty trash
`useDecryptedFile(file)`	Download + decrypt into a Blob URL (revoked on unmount); `saveBlob()` helper

Stores (`src/stores`, Zustand)

Store	State
`useAuth`	`user` , `accessToken` , `masterKey` (memory only!), `pending` sign-up; `setSession` , `clear` , `activatePending`
`useUI`	toasts, upload queue, upload modal, view mode (grid/list), search text, mobile sidebar
`useDialogs`	active dialog `{type, props}` and the file shown in the details panel
`useTheme`	`dark` / `light` , persisted in localStorage, toggles `html.light`

Crypto & API modules

Module	Purpose
`crypto/core.js`	KDF (Argon2id/PBKDF2), AES-GCM seal/open, key wrapping, chunk encryption, recovery-key encoding, account bootstrap, share-key derivation. Runs in browser and Node.
`crypto/vault.js`	`uploadFile`, `downloadFile`, `decryptNames`, `createShare`, `openShareMeta`, `downloadShare` — take an `http` adapter so tests and the seed use the same code
`api/http.js`	`api()` JSON helper with silent token refresh; `http` adapter; `refreshSession()`
`api/auth.js`	`register`, `login`, `verify2fa`, `logout`, `changePassword`, `recoverAccount`, `authKeyFor`
`lib/format.js`	`formatBytes`, `timeAgo`, `formatDate`, `fileKind`, `categoryOf`, `initials`

Development guide

Prerequisites

- Node.js **20+** and npm
- (optional) a MongoDB **replica set** — e.g. a free Atlas cluster. Not needed locally: the API starts an in-memory replica set.
- (optional) Google Chrome/Chromium for the Playwright e2e test.

Setup

```
git clone <repo> securevault && cd securevault
cd server && npm install && cp .env.example .env     # postinstall runs `prisma generate`
cd ../client && npm install
```

Running

Command	Where	What
`npm start`	server	API on `PORT` (5100)
`SEED_DEMO=1 npm start`	server	API + seed `demo@securevault.app / Demo@12345`
`npm run dev`	server	API with `node --watch`
`npm run seed`	server	Seed the demo account into a running API (`API_URL`)
`npm run dev`	client	Vite dev server on 5174, proxies `/api` → 5100
`npm run build` / `npx vite build`	client	Production build to `client/dist`

Prisma

The schema lives in `server/prisma/schema.prisma` (MongoDB provider).

```
cd server
npx prisma generate      # after any schema change (also runs on npm install)
npx prisma db push       # sync indexes to the database in DATABASE_URL (Atlas)
npx prisma studio        # data browser
npx prisma format        # format the schema
```

- There are no migrations with MongoDB; `db push` creates collections' unique indexes.
- In local in-memory mode, `config/db.js` sets `DATABASE_URL` to the `MongoMemoryReplSet` URI and runs `prisma db push --skip-generate` automatically before creating the `PrismaClient`.
- Prisma imports are done through `import { prisma } from '../config/db.js'` (an ESM live binding that is set once `connectDB()` runs).

Environment variables (`server/.env`)

Var	Default	Notes
`PORT`	5100	
`DATABASE_URL`	*(empty → in-memory)*	MongoDB replica-set URI (`MONGO_URI` also accepted)
`CLIENT_ORIGIN`	http://localhost:5174	comma-separated allow-list for CORS/CSRF
`JWT_SECRET`, `SERVER_ENC_KEY`	random per boot in dev	**required in production**
`ACCESS_TTL_SEC`	900	access token lifetime
`REFRESH_TTL_DAYS`	30	
`DEFAULT_QUOTA_BYTES`	10 GiB	per new user
`STORAGE_DRIVER`	local	`local` or `s3`
`LOCAL_STORAGE_DIR`	server/storage	
`S3_BUCKET`, `S3_REGION`, `S3_ENDPOINT`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`		for S3/R2
`AUTH_RATE_LIMIT`	20	auth attempts / 15 min / IP+email
`LOG_LEVEL`	info	pino level

Client: `VITE_API_PROXY` changes the dev proxy target.

Testing

```
cd server && npm test        # Jest + Supertest (ESM via --experimental-vm-modules)
cd client && npm run e2e     # needs API on 5100 + Vite on 5174
cd client && node tests/e2e/screenshots.mjs   # regenerate docs/screenshots (demo account
must exist)
```

Server tests (`server/tests`, 69 tests): each file starts its own in-memory replica set via `tests/helpers.js`, registers users with the *real client crypto* (`client/src/crypto`) and fast Argon2 params, and drives the API through a supertest-backed `http` adapter.

File	Covers
`auth.test.js`	register, duplicate, validation, NoSQL injection, prelogin anti-enumeration, login, lockout, rate-limit 429, refresh rotation + reuse detection, CSRF, logout, profile, password change (same MK), recovery, delete account, helmet/CORS/body limits
`twofa.test.js`	setup/QR, enable, login requires 2FA, bad ticket, backup codes single-use, disable, encrypted TOTP secret
`files.test.js`	multi-chunk round trip, ciphertext-only storage, empty files, names, quota, chunk validation, folders (nest, rename, move, cycles, trash/restore/permanent), trash/empty, versions, authorization matrix, devices, activity
`share.test.js`	public link, wrong key, password, download limit, expiry, revoke, listing, cross-user, ticket check

E2E (`client/tests/e2e/e2e.mjs`): Chromium registers a new user, saves the recovery key, uploads a 10 MiB (2-chunk) binary + text file, previews, downloads & compares SHA-256, creates a password share,

opens it in a fresh browser context (wrong then right password), compares SHA-256, navigates every page, reload → unlock, and fails on any console error.

Linting

```
cd server && npm run lint    # ESLint 8 (.eslintrc.json)
cd client && npm run lint    # ESLint 9 flat config (react, hooks, refresh)
```

Conventions

- ES modules everywhere; small single-purpose modules with JSDoc headers explaining *why*.
- Every route: `validate({ body/params/query })` with zod + ownership helper.
- Never log or persist keys, passwords or file contents.
- Suggested CI: `npm ci`, `npm run lint`, `npm test`, `npx vite build`, `npm audit --audit-level=high`.

Deployment

Recommended free/cheap setup: **MongoDB Atlas** (database) + **Cloudflare R2** (encrypted chunks) + **Render or Railway** (API) + **Vercel** (front end).

1. MongoDB Atlas

1. Create a free M0 cluster (it's a replica set — required by Prisma).
2. Database Access → add a user with a strong password. Network Access → allow your API host (or `0.0.0.0/0` with a strong password).
3. Copy the URI and add a database name: `mongodb+srv://user:pass@cluster0.xxxx.mongodb.net/securevault?retryWrites=true&w=majority`.
4. Create indexes once: `cd server && DATABASE_URL="<uri>" npx prisma db push`.

2. Cloudflare R2 (or AWS S3)

1. Create a bucket, e.g. `securevault-chunks` (keep it **private**).
2. Create an API token with Object Read & Write for that bucket → Access Key ID + Secret.
3. Endpoint: `https://<ACCOUNT_ID>.r2.cloudflarestorage.com`, region `auto`.
4. **CORS** on the bucket (browser uploads directly via pre-signed URLs):

```
[{ "AllowedOrigins": ["https://your-app.vercel.app"], "AllowedMethods": ["GET", "PUT"],
   "AllowedHeaders": ["content-type"], "MaxAgeSeconds": 3600 }]
```

For AWS S3 use the same CORS, omit `S3_ENDPOINT`, and set `S3_REGION` (e.g. `eu-central-1`).

3. API on Render / Railway

- Root directory: `server` · Build: `npm ci` (runs `prisma generate`) · Start: `npm start` · Node 20.
- Environment:

```
NODE_ENV=production
DATABASE_URL=mongodb+srv://…/securevault
CLIENT_ORIGIN=https://your-app.vercel.app
JWT_SECRET=<64 random hex>
SERVER_ENC_KEY=<64 random hex>
STORAGE_DRIVER=s3
S3_BUCKET=securevault-chunks
S3_REGION=auto
S3_ENDPOINT=https://<ACCOUNT_ID>.r2.cloudflarestorage.com
S3_ACCESS_KEY_ID=…
S3_SECRET_ACCESS_KEY=…
```

Generate secrets: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`. `app.set('trust proxy', 1)` is already set so rate limits see real client IPs.

4. Front end on Vercel

The client calls the API at the relative path `/api/v1`, so put both on **one origin** by rewriting `/api` to the API. This keeps the refresh cookie first-party and `SameSite=Strict`.

`client/vercel.json`:

```
{
  "rewrites": [
    { "source": "/api/:path*", "destination": "https://your-api.onrender.com/api/:path*" },
    { "source": "/(.*)", "destination": "/index.html" }
  ],
  "headers": [{ "source": "/(.*)", "headers": [
    { "key": "Content-Security-Policy", "value": "default-src 'self'; script-src 'self'
'wasm-unsafe-eval'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; media-src
'self' blob:; frame-src blob:; connect-src 'self' https://*.r2.cloudflarestorage.com; object-
src 'none'; base-uri 'none'; frame-ancestors 'none'" },
    { "key": "Referrer-Policy", "value": "no-referrer" },
    { "key": "X-Content-Type-Options", "value": "nosniff" },
    { "key": "Strict-Transport-Security", "value": "max-age=63072000; includeSubDomains;
preload" }
  ]}]
}
```

Vercel settings: root `client`, build `npm run build`, output `dist`. `'wasm-unsafe-eval'` is needed for Argon2id (WebAssembly). `Referrer-Policy: no-referrer` keeps share URLs out of referrers (the `#k=` fragment is never sent anyway).

5. Seed (optional)

`API_URL=https://your-app.vercel.app npm run seed` (from `server/`) creates the demo account — skip in real production.

6. Checklist

- [ ] HTTPS everywhere (automatic on Vercel/Render/Railway)
- [ ] `NODE_ENV=production` (secure cookies, HSTS, required secrets)
- [ ] Unique `JWT_SECRET` and `SERVER_ENC_KEY` (rotating `SERVER_ENC_KEY` invalidates stored TOTP secrets)
- [ ] Private bucket with CORS limited to your domain
- [ ] Atlas IP allow-list & backups enabled
- [ ] `npm audit` clean, uptime monitor on `/api/v1/health`
- [ ] For several API instances, move rate limiting to Redis (`rate-limit-redis`)

SecureVault user guide

Create your vault

1. Open SecureVault and click **Get started free** (or **Create your vault**).
2. Enter your name, email and a **master password**. The meter turns green when it's strong (12+ characters, numbers, symbols, upper & lowercase).
3. Tick "I understand SecureVault can't reset my password" and click **Create secure vault**.
4. **Save your recovery key.** Download the `.txt` or copy it somewhere safe and offline (password manager, printed paper). It's the *only* way back in if you forget your password.

   Why can't SecureVault reset my password? Your files are encrypted with a key derived from your password, in your browser. We never have that key, so nobody — not even us — can decrypt your files without your password or recovery key.

Log in on any computer

1. Go to **Log in**, enter email and master password.
2. If 2FA is on, type the 6-digit code from your authenticator app (or a backup code).
3. If you reload the page, your vault locks automatically; enter your password to **Unlock**.

Upload files

- Click **Upload** and drop files into the box or **Browse files**, or
- drag files from your desktop **anywhere** onto the page — they go into the folder you're viewing. Each file is encrypted on your device in 8 MB pieces; you'll see "Encrypting…", chunk progress and **Done**.

Organise

- **New folder** creates a folder in the current location; open it by clicking.
- Drag a file card onto a folder to move it, or use **… → Move to…**.
- **… → Rename**, **Add to favorites** (⭐ shows in **Favorites**).
- **Recent** shows recently changed files. Use the **search box** to find files by name — search happens on your device.
- Switch between **grid** and **list** view at the top right.

Preview & download

Click a file to open the details panel: images, PDFs, videos, audio and text files are decrypted and previewed right in your browser. Click **Download** to save the decrypted file.

Versions

In the details panel, **New version** uploads an updated copy while keeping the old one. Under **Version history** you can download any version or **Restore** it as the current one.

Share a file

1. Open the file → **Share**.
2. Optional: turn on **Password protection**, choose an **expiry** (1 hour … Never), a **download limit** and a private label.
3. Click **Create link**, then **Copy**. Send the full link (including the part after `#`) and, separately, the password.
4. The recipient opens the link, enters the password and the file is decrypted in *their* browser.
5. See and **revoke** all links under **Shared**. Links stop working automatically when they expire or reach the limit.

   We can't show a link again later, because its key is only in the URL you copied. Just create a new one.

Trash

Move to trash hides a file or folder. In **Trash** you can **Restore** or **Delete forever**; **Empty trash** erases everything there and frees storage.

Settings

- **Profile:** change your display name.
- **Security:** change your master password (other devices are signed out); set up **two-factor authentication** — scan the QR code, enter the code, and save your 10 backup codes; disable it with a code.
- **Devices:** see where you're signed in and **Sign out** any other device instantly.
- **Activity:** every sign-in (with "Sign-in alert"), failed attempt, upload, share and security change.
- **Appearance:** dark or light theme (also the ☀/🌙 button at the top).
- **Danger zone:** permanently delete your account and all files.

Forgot your password?

On the login page click **Use recovery key**, enter your email, the recovery key and a new password. Your files stay intact; all devices are signed out.

Demo account

Email `demo@securevault.app`, password `Demo@12345` (local demo only).